

คู่มือส่งมอบและใช้งาน WashiOS FlightStack

เอกสารนี้จัดทำขึ้นสำหรับผู้ใช้ในห้องปฏิบัติการที่ต้องการนำ WashiOS FlightStack ไป build, flash และทดสอบกับบอร์ดจริงด้วยตนเอง โดยไม่จำเป็นต้องรู้รายละเอียดภายในของระบบมาก่อน

1. ภาพรวม

WashiOS FlightStack คือชุดซอฟต์แวร์ฝังตัวสำหรับบอร์ด STM32 ที่สร้างอยู่บน FreeRTOS โดยเพิ่มส่วนประกอบที่จำเป็นต่อการทดลองระบบที่ต้องการความน่าเชื่อถือ เช่น bootloader ที่ตรวจสอบ firmware ก่อนเริ่มทำงาน, runtime core ที่มี task health และ watchdog, รวมถึง demo payload สำหรับทดสอบการสื่อสารระหว่างบอร์ดจริงสองตัว

ระบบนี้แบ่งเป็น 3 ส่วนหลัก:

- bootloader/ หรือ WashiBoot: firmware ตัวแรกที่รันหลัง reset ใช้ตรวจสอบ application ก่อน jump เข้า core
- core/ หรือ WashiOS-Core: application หลักที่รัน FreeRTOS task, watchdog, telemetry, laser communication และ payload link
- demo-payload/: firmware สำหรับบอร์ด payload responder อีกตัวหนึ่ง ใช้ตอบกลับบอร์ดหลักผ่าน UART

ลำดับการทำงานโดยรวม:

```
Power on / Reset
-> WashiBoot bootloader
-> ตรวจสอบ application image ใน Slot A หรือ Slot B
-> ถ้า image ถูกต้องจึง jump เข้า WashiOS-Core
-> WashiOS-Core เริ่ม FreeRTOS และรัน tasks
-> tasks ทำงาน เช่น heartbeat, watchdog, payload link, laser telemetry
```

2. โครงสร้าง Repository

หลังจัดเป็น monorepo แล้ว โครงสร้างหลักคือ:

```
WashiOS/
  README.md
  bootloader/
  core/
  demo-payload/
  docs/
  tools/
```

ความหมายของแต่ละโฟลเดอร์:

โฟลเดอร์	หน้าที่
bootloader/	โค้ด WashiBoot, linker script, unit test และ provisioning script
core/	WashiOS-Core, FreeRTOS, HAL abstraction, app tasks, protocol และ SITL tests
demo-payload/	firmware สำหรับบอร์ด NUCLEO-G474RE ที่จำลอง payload responder
docs/	เอกสารคู่มือ, handoff สำหรับ research และ provisioning note
tools/	PowerShell scripts สำหรับ build, flash และ test

3. Hardware ที่ต้องใช้

อุปกรณ์	ใช้ทำอะไร
ST NUCLEO-G431RB	บอร์ดหลัก หรือ OBC ใช้รัน bootloader และ core
ST NUCLEO-G474RE	บอร์ด payload responder สำหรับ demo payload
สาย USB	ใช้จ่ายไฟและ flash ผ่าน ST-LINK
สาย jumper	ใช้ต่อ UART ระหว่างบอร์ด
laser module หรือ laser driver	ใช้สำหรับ laser communication demo
oscilloscope หรือ logic analyzer	ใช้ตรวจ PA6, UART และ pulse timing

ข้อควรระวัง: ห้ามต่อ laser diode เข้ากับ GPIO โดยตรง ให้ใช้ laser driver, transistor, MOSFET หรือ laser module ที่รับ logic 3.3 V ได้

4. Software ที่ต้องติดตั้ง

- Git
- Python
- Visual Studio Code
- PlatformIO Core หรือ PlatformIO Extension
- ST-LINK driver

ตรวจสอบ PlatformIO:

```
pio --version
```

ถ้า PowerShell หา pio ไม่เจอ ให้เปิด terminal จาก PlatformIO IDE หรือเพิ่ม PlatformIO เข้า PATH

5. Memory Map

สำหรับ STM32G431RB:

ส่วน	Address	Size
Bootloader	0x08000000	16 KiB
Slot A	0x08004000	56 KiB
Slot B	0x08012000	56 KiB
RAM	0x20000000	32 KiB

เมื่อ reset บอร์ด MCU จะเริ่มที่ bootloader ก่อน จากนั้น bootloader จะตรวจ application ใน Slot A หรือ Slot B ถ้า image ถูกต้องจึง jump เข้า core

6. Pin Mapping สำคัญ

สำหรับ NUCLEO-G431RB:

Pin	หน้าที่
PA5	Heartbeat LED และ bootloader safe-loop beacon
PA6	Laser TX GPIO
PA2	USART2_TX debug console
PC4	USART1_TX payload UART
PC5	USART1_RX payload UART

สำหรับ laser communication:

PA6 -> input ของ laser driver
GND -> ground ร่วมกับวงจร laser

7. วิธี Clone และเตรียม Project

clone repository:

```
git clone https://github.com/NatdanaiHS/WashiOS.git
cd WashiOS
```

ตรวจว่าเห็นโครงสร้าง:

```
bootloader
core
demo-payload
docs
tools
README.md
```

8. หลักสำคัญก่อน Build และ Flash

ต้องจำกฎนี้เสมอ:

ต้อง build core ก่อน build bootloader

เหตุผลคือ bootloader จะอ่าน ELF ของ core, ตรวจสอบ vector table, คำนวณ CRC-32 แล้วฝังค่า CRC ลงใน build ของ bootloader ถ้าแก้ core แล้วไม่ rebuild bootloader ค่า CRC จะไม่ตรงและ bootloader อาจไม่ยอม boot application

ลำดับที่ถูกต้อง:

1. build core environment ที่ต้องการ
2. build bootloader environment ที่ตรงกัน
3. upload bootloader
4. upload core application
5. reset board

9. ทดสอบ Native Tests

จาก repository root:

```
.\tools\test_native.ps1
```

หรือรันเอง:

```
cd bootloader  
pio test -e native
```

```
cd ..\core  
pio test -e native
```

10. ใช้งาน Laser Communication Demo

โมดูลนี้ใช้สำหรับทดสอบการส่งข้อมูลผ่าน laser ที่ถูกควบคุมด้วย GPIO

environment ที่ใช้:

```
nucleo_g431rb_lasercom
```

build และ flash แบบเร็ว:

```
cd <repo-root>  
.\tools\flash_lasercom.ps1
```

ถ้าต้องการ build อย่างเดียว:

```
.\tools\build_lasercom.ps1
```

คำสั่ง manual:

```
cd core
pio run -e nucleo_g431rb_lasercom

cd ../bootloader
pio run -e nucleo_g431rb_lasercom
pio run -e nucleo_g431rb_lasercom -t upload

cd ../core
pio run -e nucleo_g431rb_lasercom -t upload
```

สิ่งที่ควรเห็น:

- PA5 หรือ LD2 กระพริบจาก heartbeat
- PA6 มี pulse สำหรับควบคุม laser
- ถ้าวัดด้วย oscilloscope จะเห็น sync pulse และ pulse สั้น/ยาวของข้อมูล

รูปแบบ pulse ปัจจุบัน:

Bit	High time	Low gap
0	2000 us	2000 us
1	4000 us	2000 us

ไฟล์ที่เกี่ยวข้อง:

```
core\include\comms\LaserPdmTx.hpp
core\src\app\LaserTelemetryTask.hpp
```

ข้อความ ASCII test ปัจจุบันอยู่ใน LaserTelemetryTask.hpp หากแก้ไขข้อความ ต้อง flash ใหม่ด้วย
.\tools\flash_lasercom.ps1

11. ใช้งาน Payload UART Demo

Payload UART demo ใช้บอร์ดสองตัว:

ฝั่ง	บอร์ด	โฟลเดอร์	หน้าที่
OBC	NUCLEO-G431RB	core/ + bootloader/	ส่ง poll และตรวจ response
Payload	NUCLEO-G474RE	demo-payload/	รับ poll และตอบ telemetry

environment ที่ใช้:

```
OBC core/bootloader = nucleo_g431rb_payload_demo
Payload responder = nucleo_g474re
```

flash ฝั่ง OBC G431:

```
cd <repo-root>
.\tools\flash_payload_demo.ps1
```

flash ฝั่ง payload responder G474:

```
cd <repo-root>
.\tools\build_payload_responder.ps1
.\tools\flash_payload_responder.ps1
```

คำสั่ง manual สำหรับ payload responder:

```
cd demo-payload
pio run -e nucleo_g474re
pio run -e nucleo_g474re -t upload
```

ถ้าเสียบ ST-LINK สองบอร์ดพร้อมกัน ให้ระวัง flash ผิดบอร์ด ช่วงเริ่มต้นควรเสียบและ flash ที่ละบอร์ด

12. Wiring ระหว่าง G431 และ G474

ต่อแบบ cross TX/RX:

G431 OBC	G474 Payload
D1 / PC4 / USART1_TX	D0 / PC5 / USART1_RX
D0 / PC5 / USART1_RX	D1 / PC4 / USART1_TX
GND	GND

ให้จ่ายไฟทั้งสองบอร์ดด้วย USB ของตัวเอง และอย่าต่อ 3V3 หรือ 5V ของสองบอร์ดเข้าหากัน

13. Serial Monitor

ฝั่ง OBC G431:

```
USART1 PC4/PC5 = คุยกับ payload
USART2 PA2      = debug console ผ่าน ST-LINK VCP
baud rate      = 115200
```

ฝั่ง payload G474:

```
USART1 PC4/PC5 = payload link
LPUART1 PA2/PA3 = debug console ผ่าน ST-LINK VCP
baud rate      = 115200
```

ดู COM port:

```
pio device list
```

เปิด monitor:

```
pio device monitor -p COM_OBC -b 115200
pio device monitor -p COM_PAYLOAD -b 115200
```

ให้แทน COM_OBC และ COM_PAYLOAD ด้วย COM port จริง

14. Fault Mode ของ Payload Responder

กด user button บน G474 เพื่อเปลี่ยนโหมด:

NORMAL -> SILENT -> BAD_CRC -> DELAYED -> NORMAL

Mode	ความหมาย
NORMAL	ตอบ telemetry ถูกต้อง
SILENT	รับ poll แต่ไม่ตอบ
BAD_CRC	ตอบกลับแต่ทำ CRC ให้ผิด
DELAYED	ตอบช้า 250 ms เกิน deadline 100 ms ของ OBC

LED บน G474 จะ toggle ทุกครั้งที่ได้รับ poll request ที่ถูกต้อง

สิ่งที่ควรเห็นใน console:

```
[PAYLOAD] READY board=NUCLEO-G474RE mode=NORMAL baud=115200
[PAYLOAD] LINK_ACTIVE
[PAYLOAD] MODE=SILENT
[PAYLOAD] MODE=BAD_CRC
[PAYLOAD] MODE=DELAYED
```

ฝั่ง OBC ควรเห็นสถานะเช่น:

```
[OBC] PAYLOAD_LINK_START baud=115200
[OBC] PAYLOAD_ONLINE ...
[OBC] PAYLOAD_TIMEOUT consecutive=...
[OBC] PAYLOAD_OFFLINE consecutive=...
[OBC] PAYLOAD_RECOVERED ...
```

payload offline ไม่ได้แปลว่า OBC firmware ดาย เพราะ PayloadLinkTask ยัง check-in กับ watchdog ได้อยู่ถ้า task ยังทำงานปกติ

15. Payload Protocol

OBC ส่ง PollRequest ทุกประมาณ 500 ms และ payload ต้องตอบ TelemetryResponse กลับมาด้วย sequence เดิม

frame มีขนาด 32 bytes:

Offset	Size	Field
0	1	sync0 = 0x57
1	1	sync1 = 0x50
2	1	version = 1
3	1	message type
4	4	sequence, little-endian
8	2	payload length, little-endian
10	2	flags, little-endian
12	16	payload
28	4	CRC-32 over bytes 0..27

message type:

Type	ค่า
PollRequest	0x01
TelemetryResponse	0x81

ถ้า CRC ผิด OBC จะ reject frame เป็น CRC error ถ้า sequence ผิดจะ reject เป็น sequence error
ถ้าไม่ตอบภายในเวลา OBC จะนับ timeout และถ้า timeout 3 ครั้งติดกันจะมองว่า payload offline

16. Bootloader ทำอะไร

bootloader/ หรือ WashiBoot เป็น firmware ตัวแรกหลัง reset หน้าที่หลักคือ:

- 1. ตรวจสอบ boot metadata
- 2. ตรวจสอบ retained fault log
- 3. ตรวจสอบ active slot
- 4. ตรวจสอบ vector table ของ application
- 5. ตรวจสอบ CRC-32 ของ application
- 6. ถ้าผ่านจึง jump เข้า application
- 7. ถ้า fail จะลอง fallback slot
- 8. ถ้าไม่มี slot ที่ใช้ได้ จะเข้า safe loop

safe loop คือสถานะที่ระบบไม่ jump เข้า application และใช้ PA5 toggle เป็น beacon เพื่อบอกว่าระบบอยู่ในสถานะปลอดภัย

17. Troubleshooting

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
build bootloader แล้วหา core ELF ไม่เจอ	ยังไม่ได้ build core environment ที่ตรงกัน	build core ก่อน หรือใช้ script ใน tools/
upload core แล้วไม่รัน	CRC ไม่ตรง หรือยังไม่มี bootloader	ใช้ .\tools\flash_lasercom.ps1 หรือ .\tools\flash_payload_demo.ps1
PA6 ไม่มี pulse	ใช้ env ผิด หรือ bootloader ไม่ jump เข้า core	ตรวจสอบ env, PA5 heartbeat, และ build/flash ตามลำดับ
laser ไม่ยิง แต่ PA6 มี pulse	hardware laser driver ต่อดiodeหรือไม่รับ 3.3 V	ตรวจสอบ driver, GND, supply และ enable pin
payload ไม่ตอบ	ต่อ TX/RX ไม่ไขว้, ไม่ต่อ GND, COM/board ผิด	ตรวจสอบ wiring และ flash ที่ละบอร์ด
serial monitor ไม่เห็นข้อความ	เลือก COM ผิดหรือ baud ผิด	ใช้ pio device list และ baud 115200

18. Checklist ทดสอบ LaserCom

- clone repo แล้วเห็น bootloader/, core/, demo-payload/, docs/, tools/
- ติดตั้ง PlatformIO แล้ว
- ใช้บอร์ด NUCLEO-G431RB
- ต่อ PA6 ไปที่ input ของ laser driver ไม่ต่อ laser diode ตรง
- ต่อ GND ร่วม

- run `.\tools\flash_lasercom.ps1` สำเร็จ
- กด reset
- PA5 heartbeat กระพริบ
- PA6 มี pulse
- oscilloscope เห็น sync pulse และ pulse 2 ms / 4 ms

19. Checklist ทดสอบ Payload UART

- flash OBC G431 ด้วย `.\tools\flash_payload_demo.ps1`
- flash payload G474 ด้วย `.\tools\flash_payload_responder.ps1`
- ต่อ PC4/PC5 แบบ cross TX/RX
- ต่อ GND ร่วม
- ไม่ต่อ 3V3/5V ระหว่างบอร์ด
- เปิด serial monitor ที่ baud 115200
- เห็น payload READY
- เห็น OBC PAYLOAD_ONLINE
- กด button บน G474 แล้ว mode เปลี่ยน NORMAL/SILENT/BAD_CRC/DELAYED
- กลับมา NORMAL แล้ว OBC รายงาน recovery

20. ข้อจำกัดของระบบปัจจุบัน

- CRC-32 ใช้ตรวจ corruption แต่ไม่ใช่ security
- ยังไม่มี signed firmware verification
- ยังไม่มี anti-rollback
- boot metadata ยังเหมาะกับ development/prototype มากกว่า production
- LaserCom ยังเป็น transmitter demo ไม่ใช่ optical communication stack สมบูรณ์
- payload demo เป็น hardware-in-the-loop prototype ไม่ใช่ payload subsystem จริง
- ยังควรเพิ่ม fault-injection experiment และ measurement ก่อนใช้เป็น paper ที่แข็งแรง

21. สรุป

ถ้าต้องการทดสอบ laser communication:

```
cd <repo-root>
.\tools\flash_lasercom.ps1
```

ถ้าต้องการทดสอบ payload UART demo:

```
cd <repo-root>
.\tools\flash_payload_demo.ps1
.\tools\flash_payload_responder.ps1
```

สิ่งที่ต้องจำที่สุดคือ build core ก่อน bootloader เสมอ เพราะ bootloader ต้องใช้ CRC ของ core firmware ล่าสุด